

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada a Objetos
Catedrático/a: Erick Marroquín



Ejercicio 3 - Arreglos dinámicos, ArrayList, List y Excepciones.

Nombre: Javier Alejandro Apen Solis
Carné: 26864
10/08/2026

Contenido

Contenido	2
1. Análisis	3
1.1 Clases identificadas y rol MVC	3
1.2 Propósito de los atributos y de la colección dinámica	3
1.3 Relaciones entre clases	4
2. Tabla de Atributos.....	5
OrdenServicio	5
OrdenController	5
OrdenView.....	5
Main	5
3. Tabla de Métodos.....	6
OrdenServicio	6
OrdenController	6
OrdenView	7
Main	8
4. Diseño	8

1. Análisis

1.1 Clases identificadas y rol MVC

Clase	Rol MVC	Responsabilidad
OrdenServicio	Model	Representa una orden de servicio individual y encapsula sus datos (número, propietario, placa, descripción, costo).
OrdenController	Controller	Administra la colección dinámica de órdenes; contiene toda la lógica de negocio: registrar, buscar, modificar, cancelar y generar reportes.
OrdenView	View	Maneja toda la entrada y salida con el usuario: muestra el menú, captura los datos y presenta resultados y mensajes de error.
Main	Driver	Crea las instancias de OrdenController y OrdenView, y arranca la ejecución del programa.

1.2 Propósito de los atributos y de la colección dinámica

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección ordenes (`List<OrdenServicio>` dentro de `OrdenController`) almacena objetos completos de tipo `OrdenServicio`, no datos sueltos. Cada objeto agrupa el número de orden, el propietario, la placa, la descripción del servicio y el costo estimado, de forma que cada elemento de la lista representa una orden de servicio activa en el taller.

5. ¿Dónde utilizará `List` y dónde realizará la creación mediante `ArrayList`?

El tipo declarado del atributo y de los métodos que exponen la colección es `List<OrdenServicio>`, programando contra la interfaz. La instanciación concreta se realiza una única vez, en el constructor de `OrdenController`: `this.ordenes = new ArrayList<>()`. De esta forma toda la clase trabaja con la referencia `List`, aunque internamente la estructura sea un `ArrayList`.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

`OrdenServicio` no tiene valores por defecto: todos sus atributos se reciben obligatoriamente por parámetro en el constructor, y el propio constructor valida que los campos de texto no estén vacíos y que el costo sea mayor que 0, lanzando `IllegalArgumentException` si no se cumple. Al crear `OrdenController`, la lista `ordenes` inicia vacía (`new ArrayList<>()`); el sistema no precarga órdenes de ejemplo, ya que el enunciado no lo requiere.

9. ¿Cómo identificará una orden específica dentro de la colección?

El atributo `numeroOrden` funciona como llave única de cada orden. Para localizar una orden específica se recorre la lista con un bucle (`for-each` o `for indexado`) comparando `orden.getNumeroOrden() == numeroBuscado` hasta encontrar la coincidencia; si se recorre toda la lista sin encontrarla, se lanza una `NoSuchElementException`.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Para agregar: antes de crear el objeto se recorre la lista para verificar que el número de orden no exista; si ya existe se lanza `IllegalArgumentException`, y si no, se agrega con `ordenes.add(nuevaOrden)`. Para eliminar: se busca primero la orden con el número indicado (reutilizando el método de búsqueda); si existe se elimina con `ordenes.remove(ordenEncontrada)`, y si no existe se lanza `NoSuchElementException` en vez de intentar eliminar.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se recorre toda la lista con un bucle `for-each` comparando `orden.getPlaca().equalsIgnoreCase(placaBuscada)`, y cada coincidencia se agrega a una nueva `List<OrdenServicio>` que se retorna al finalizar el recorrido. Esta lista puede contener cero, una o varias órdenes, dependiendo de cuántas veces haya ingresado el mismo vehículo.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Con un bucle for-each se recorre la lista completa acumulando el costoEstimado de cada orden en una variable double (valor total). El costo promedio se obtiene dividiendo ese total entre ordenes.size(). Antes de dividir se valida que la lista no esté vacía; si lo está, se lanza IllegalStateException para evitar una división entre cero.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se recorre la lista una sola vez manteniendo una referencia OrdenServicio mayor, inicializada con el primer elemento; en cada iteración, si orden.getCostoEstimado() es mayor al de mayor, se actualiza la referencia. Si la colección está vacía se lanza IllegalStateException en lugar de retornar null, evitando así un NullPointerException posterior.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Principalmente cuatro: (1) el usuario ingresa texto no numérico donde se espera un número entero o decimal (número de orden o costo), lo que provoca una NumberFormatException al usar Integer.parseInt o Double.parseDouble; (2) se intenta registrar un número de orden que ya existe o datos vacíos/costo inválido, lo que se controla lanzando IllegalArgumentException; (3) se intenta buscar, modificar o cancelar una orden con un número que no está registrado, lo que se controla lanzando NoSuchElementException; (4) se intenta calcular el promedio o la orden de mayor costo cuando la colección está vacía, lo que se controla lanzando IllegalStateException.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

Se protegen con try-catch: la lectura de datos numéricos desde el Scanner (número de orden y costo estimado, capturando NumberFormatException), el registro de una orden nueva (capturando IllegalArgumentException por duplicado o dato inválido), y la búsqueda, modificación y cancelación de una orden (capturando NoSuchElementException cuando el número de orden no existe). En todos los casos el catch informa al usuario del error sin detener la ejecución del programa.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

El bloque finally se utiliza en la operación de modificar orden (opción del menú OrdenView.modificarOrden): sin importar si la modificación se realizó con éxito o si se lanzó una NoSuchElementException por número inexistente, el finally imprime un mensaje de cierre como “Operación de modificación finalizada.” y asegura que el flujo del programa vuelva al menú principal. Esto es necesario porque, según el requisito 18, el sistema debe seguir funcionando después de manejar una excepción, y el finally es el mecanismo que garantiza esa continuidad sin duplicar código en el bloque try y en cada catch.

1.3 Relaciones entre clases

Relación	Tipo	Justificación
OrdenController y OrdenServicio	Composición	Las órdenes se crean y existen únicamente dentro de la colección del controlador; si el controlador se destruye, las órdenes dejan de existir.
OrdenView y OrdenController	Dependencia	La vista invoca métodos del controlador para ejecutar cada opción del menú, pero no almacena una copia de la colección de órdenes.
Main y OrdenController / OrdenView	Dependencia	El driver crea las instancias de ambas clases y las conecta, pero no forma parte de su estructura interna.

2. Tabla de Atributos

OrdenServicio

Atributo	Tipo de Dato	Modificador de acceso	Propósito
numeroOrden	int	private	Identificador único de la orden de servicio.
propietario	String	private	Nombre del propietario del vehículo.
placa	String	private	Placa del vehículo asociado a la orden.
descripcionServicio	String	private	Descripción del servicio o reparación solicitada.
costoEstimado	double	private	Costo estimado del servicio; debe ser mayor que 0.

OrdenController

Atributo	Tipo de Dato	Modificador de acceso	Propósito
ordenes	List<OrdenServicio> (instanciada como ArrayList)	private	Colección dinámica que almacena todas las órdenes de servicio activas; punto único de acceso a los datos.

OrdenView

Atributo	Tipo de Dato	Modificador de acceso	Propósito
scanner	Scanner	private	Captura la entrada del usuario desde consola.
controller	OrdenController	private	Referencia al controlador utilizada para ejecutar las operaciones solicitadas desde el menú.

Main

No aplica (clase driver, solo contiene el método main).

3. Tabla de Métodos

OrdenServicio

Método	Retorno	Modificador de acceso.	Parámetros	Propósito
OrdenServicio (constructor)	---	public	int numeroOrden, String propietario, String placa, String descripcionServicio, double costoEstimado	Inicializa una orden validando que los textos no estén vacíos y que el costo sea mayor que 0 (lanza IllegalArgumentException si no se cumple).
getNumeroOrden	int	public	---	Devuelve el número de orden.
getPropietario	String	public	---	Devuelve el nombre del propietario.
getPlaca	String	public	---	Devuelve la placa del vehículo.
getDescripcionServicio	String	public	---	Devuelve la descripción del servicio.
getCostoEstimado	double	public	---	Devuelve el costo estimado.
setDescripcionServicio	void	public	String nuevaDescripcion	Actualiza la descripción del servicio (única propiedad, junto al costo, modificable tras el registro).
setCostoEstimado	void	public	double nuevoCosto	Actualiza el costo estimado, validando que sea mayor que 0.
toString	String	public	---	Representación en texto de la orden, usada por OrdenView para mostrar los resultados.

OrdenController

Método	Retorno	Modificador de acceso.	Parámetros	Propósito
OrdenController (constructor)	---	public	---	Inicializa la colección dinámica: this.ordenes = new ArrayList<>().
registrarOrden	void	public	int numeroOrden, String propietario, String placa, String descripcion, double costo	Valida y agrega una nueva orden a la colección; lanza IllegalArgumentException si el número ya existe o algún dato es inválido.
buscarPorNumero	OrdenServicio	public	int numeroOrden	Recorre la colección y retorna la orden encontrada; lanza NoSuchElementException si no existe.
modificarOrden	void	public	int numeroOrden, String nuevaDescripcion, double nuevoCosto	Ubica la orden y actualiza su descripción y costo; lanza NoSuchElementException si no existe o IllegalArgumentException si el nuevo costo es inválido.
cancelarOrden	void	public	int numeroOrden	Ubica y elimina la orden de la colección; lanza NoSuchElementException si no existe.
consultarPorPlaca	List<OrdenServicio>	public	String placa	Recorre la colección y retorna todas las órdenes cuya placa coincide.

Método	Retorno	Modificador de acceso.	Parámetros	Propósito
listarOrdenes	List<OrdenServicio>	public	---	Retorna una copia de la colección completa para consulta general, sin exponer la referencia interna.
calcularValorTotal	double	Public	---	Suma el costo estimado de todas las órdenes activas.
calcularCostoPromedio	double	public	---	Divide el valor total entre la cantidad de órdenes; lanza IllegalStateException si la colección está vacía.
obtenerOrdenMayorCosto	OrdenServicio	public	---	Recorre la colección y retorna la orden con el costo estimado más alto; lanza IllegalStateException si está vacía.
cantidadOrdenes	int	public	---	Devuelve la cantidad de órdenes registradas actualmente (ordenes.size()).

OrdenView

Método	Retorno	Modificador de acceso	Parámetros	Propósito
OrdenView (constructor)	---	public	OrdenController controller	Recibe e inicializa la referencia al controlador y el Scanner.
iniciar	void	public	---	Ejecuta el bucle principal del menú hasta que el usuario seleccione Salir.
mostrarMenu	void	private	---	Imprime las 10 opciones del menú.
registrarOrden	void	private	---	Solicita los datos de una orden y llama a controller.registrarOrden dentro de un try-catch.
consultarOrdenes	void	private	---	Muestra todas las órdenes retornadas por controller.listarOrdenes.
buscarOrden	void	private	---	Solicita el número de orden y muestra su información o el mensaje de error si no existe.
modificarOrden	void	private	---	Solicita el número, nueva descripción y nuevo costo; usa try-catch-finally alrededor de controller.modificarOrden.
cancelarOrden	void	private	---	Solicita el número de orden y llama a controller.cancelarOrden dentro de un try-catch.
consultarPorPlaca	void	private	---	Solicita la placa y muestra todas las órdenes asociadas.
reporteCostos	void	private	---	Muestra el valor total y el costo promedio de las órdenes activas.
ordenMayorCosto	void	private	---	Muestra la información de la orden con el costo estimado más alto.
cantidadOrdenes	void	private	---	Muestra cuántas órdenes están registradas actualmente.
leerEntero	int	private	String mensaje	Solicita y valida un valor entero, capturando NumberFormatException.
leerDouble	double	private	String mensaje	Solicita y valida un valor decimal, capturando NumberFormatException.

Main

Método	Retorno	Modificador de acceso	Parámetros	Propósito
main	void	public static	String[] args	Crea la instancia de OrdenController y de OrdenView, y llama a view.iniciar() para arrancar el sistema.

4. Diseño

